PSET 3 — Data Collection and Management Exercise

Topic 5: Mental health awareness (post-COVID) Platform: Reddit — r/Suicidal_Thoughts

---

How to read this notebook

The subreddit archive gives us two linked datasets. We analyse them as **two halves of one conversation**:

	Dataset	Lead	What it tells us
Help-seeking	9,410 submissions	Mostafa	What people in crisis *say*
Help-giving	33,206 comments	Zuri	How the community *responds*

Every comment carries a `link_id` pointing back to the post it replies to, so the two halves join on a real key. That join is the spine of this notebook: it lets us ask not just *"what do people post?"* and *"what do people reply?"* but **"does the reply arrive at all?"**

Section map (mirrors the submission rubric)

1. **Data Collection** — provenance, anonymisation, sampling frame → 200-item analysis sample
2. **Data Examination** — overview, temporal patterns, language, the join, data quality
3. **Ethical Analysis** — the five required questions
4. **Conclusion**

A note on the code

We use **pandas + an explicit, inline stopword list** rather than sklearn's `ENGLISH_STOP_WORDS`. That list is version-dependent and silently splits contractions, which is why an earlier draft of this analysis returned "don", "ve" and "ll" as top words. Defining the stoplist in the notebook makes the word-frequency results auditable and exactly reproducible by anyone who re-runs this file.

0. Setup

The two `.zst` archives live on our separate Google Drives, so the paths below are the **only lines either of us needs to edit**. Everything downstream is shared.

```
# ── EDIT THESE TWO PATHS ONLY ──────────────────────────────────────────
SUBMISSIONS_PATH = "Suicidal_Thoughts_submissions.zst"   # Mostafa's Drive
COMMENTS_PATH    = "Suicidal_Thoughts_comments.zst"      # Zuri's Drive
DRIVE_PREFIX     = "/content/drive/MyDrive/"             # set to "" to run locally
# ───────────────────────────────────────────────────────────────────────

import io, json, os, re, hashlib, datetime as dt
from collections import Counter
import pandas as pd, numpy as np
import matplotlib.pyplot as plt

try:                                    # Colab
    from google.colab import drive
    drive.mount("/content/drive")
    import subprocess, sys
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "zstandard"], check=False)
except Exception:                       # local
    DRIVE_PREFIX = ""

SUB_FILE = os.path.join(DRIVE_PREFIX, SUBMISSIONS_PATH)
COM_FILE = os.path.join(DRIVE_PREFIX, COMMENTS_PATH)

# WHO declared COVID-19 a pandemic on this date — our pre/post boundary.
CUTOFF = pd.Timestamp("2020-03-11")

pd.set_option("display.width", 200)
plt.rcParams["figure.dpi"] = 110
print("Submissions :", SUB_FILE)
print("Comments    :", COM_FILE)
print("Era cutoff  :", CUTOFF.date(), "(WHO pandemic declaration)")
```

```
Mounted at /content/drive
Submissions : /content/drive/MyDrive/Suicidal_Thoughts_submissions.zst
```

```
Comments    : /content/drive/MyDrive/Suicidal_Thoughts_comments.zst
Era cutoff  : 2020-03-11 (WHO pandemic declaration)
```

---

1. Data Collection

1.1 Platform and topic — and why

We chose **Reddit**, and specifically **r/Suicidal_Thoughts**, for three reasons.

Pseudonymity. Unlike Twitter/X or Instagram, Reddit accounts are not tied to real names. People discussing suicidal ideation self-select onto Reddit *precisely because* it is pseudonymous. For a topic this sensitive, that is not incidental — it is the reason the data exists at all, and it materially lowers (though does not eliminate) re-identification risk.

Legitimate access. We did not scrape. We used the **Pushshift/Academic Torrents monthly archive dumps**, a de facto standard corpus in computational social science. Scraping Reddit at this volume would breach the platform's Terms of Service; the archive dumps are distributed for research use, which keeps us inside the assignment's "comply with the platform's ToS" requirement.

It is the right instrument for the question. The brief's Topic 5 asks about mental health awareness *following COVID-19*. A general-purpose platform gives us people *talking about* mental health. A dedicated support subreddit gives us people *in* it — and, crucially, the community responding to them. That second half is what makes a pre/post comparison meaningful rather than merely lexical.

1.2 What we are actually claiming — and a warning we will earn later

The obvious hypothesis is *"COVID caused a surge in suicidal ideation posts."* The raw counts appear to support it overwhelmingly.

They do not. Section 2.2 shows the apparent surge is almost entirely explained by the subreddit's own audience growth. We flag this here because it is the single most important methodological finding in this notebook, and the one most likely to be got wrong.

1.3 Loading and anonymising

Anonymisation is applied **identically to both datasets** at load time, before any analysis touches the data. In an earlier draft only the submissions were hashed while the comments kept raw usernames — a contradiction inside a single notebook, on a suicidal-ideation corpus. Doing it once, in one function, makes that class of mistake impossible.

We use a **salted SHA-256 hash, truncated to 12 characters**:

- *Salted*, because an unsalted hash of a username is trivially reversible — an attacker with a list of Reddit usernames can just hash them all and look up matches. The salt makes that attack require our salt.
- *Truncated*, because we only need the pseudonyms to be distinguishable from each other, not to be cryptographically unique.
- `[deleted]` and `[removed]` collapse to a single `deleted_or_removed` sentinel. These are not a user — they are the *absence* of a user, and treating them as one account would invent a superuser with thousands of posts.

Honest limitation: hashing usernames is not anonymisation of the *text*. Post bodies on this subreddit routinely contain ages, cities, school names and family details. We return to this in the ethics section, because it is the reason we do not redistribute this data.

```
SALT = "pset3-group-salt"   # in real research this would be secret and destroyed after use

def pseudonymise(author: pd.Series) -> pd.Series:
    """Salted, truncated SHA-256. [deleted]/[removed]/null -> one shared sentinel."""
    missing = author.isna() | author.isin(["[deleted]", "[removed]"])
    hashed = author.fillna("").map(
        lambda a: "user_" + hashlib.sha256((SALT + a).encode()).hexdigest()[:12]
    )
    return hashed.mask(missing, "deleted_or_removed")


def _zst_lines(path: str):
    """Yield decompressed lines. Uses the zstandard module; falls back to the zstd CLI."""
    try:
        import zstandard as zstd
        with open(path, "rb") as fh:
            reader = zstd.ZstdDecompressor().stream_reader(fh)
            yield from io.TextIOWrapper(reader, encoding="utf-8")
    except ImportError:
        import subprocess
        proc = subprocess.Popen(["zstdcat", path], stdout=subprocess.PIPE)
        yield from io.TextIOWrapper(proc.stdout, encoding="utf-8")
```

```
def load_zst(path: str) -> pd.DataFrame:
    """Read a line-delimited JSON .zst dump into a DataFrame, then anonymise + time-tag."""
    rows, bad = [], 0
    for line in _zst_lines(path):
        line = line.strip()
        if not line:
            continue
        try:
            rows.append(json.loads(line))
        except json.JSONDecodeError:
            bad += 1
    if bad:
        print(f"  ! {bad} unparseable lines skipped")

    df = pd.DataFrame(rows)
    df["author_hash"] = pseudonymise(df["author"])
    df = df.drop(columns=["author"])                      # raw usernames never survive load
    df["created"] = pd.to_datetime(df["created_utc"].astype("int64"), unit="s")
    df["era"] = np.where(df["created"] < CUTOFF, "Pre-pandemic", "Pandemic-era")
    return df


subs = load_zst(SUB_FILE)     # help-seeking  (Mostafa)
coms = load_zst(COM_FILE)     # help-giving   (Zuri)

# the join key: a comment's link_id is "t3_<submission id>"
coms["parent_post"] = coms["link_id"].str.replace("t3_", "", regex=False)

print(f"Submissions : {len(subs):>6,} rows x {subs.shape[1]:>3} fields")
print(f"Comments    : {len(coms):>6,} rows x {coms.shape[1]:>3} fields")
print(f"Total items : {len(subs) + len(coms):>6,}")
assert "author" not in subs.columns and "author" not in coms.columns, "raw usernames leaked!"
print("\nRaw usernames successfully stripped from both datasets.")
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_911/1916139284.py in <cell line: 0>()
     45
     46
---> 47 subs = load_zst(SUB_FILE)     # help-seeking  (Mostafa)
     48 coms = load_zst(COM_FILE)     # help-giving   (Zuri)
     49

 ──────────────────────────────────────────────────  1 frames  ──────────────────────────────────────────────────
/tmp/ipykernel_911/1916139284.py in _zst_lines(path)
     14     try:
     15         import zstandard as zstd
---> 16         with open(path, "rb") as fh:
     17             reader = zstd.ZstdDecompressor().stream_reader(fh)
     18             yield from io.TextIOWrapper(reader, encoding="utf-8")

FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/Suicidal_Thoughts_submissions.zst'
```

Next steps: Explain error

1.4 Sampling frame vs. unit of analysis

The brief asks for **100–200 posts/comments**. The archive contains 42,616. We reconcile that deliberately rather than quietly:

- The **full archive is our sampling frame.** We need it whole to measure things a sample cannot see — subscriber growth, the response rate across all threads, the prevalence of automated accounts. Sampling first would have hidden every one of the data-quality findings in §2.5.
- The **unit of analysis is a 200-item sample**: 100 pre-pandemic posts + 100 pandemic-era posts, stratified by era, `random_state=42`.

Stratifying is not optional here. A simple random draw of 200 posts would return roughly **196 pandemic-era posts and 4 pre-pandemic ones** — the eras are that lopsided (204 vs 9,206). There would be nothing left to compare. Stratification forces a balanced contrast, at the cost of over-representing the pre-pandemic era ~45× relative to its true share. That is the correct trade-off for a *comparison*, and the wrong one for estimating *prevalence* — so we never use the sample to make claims about volume, only about language.

We also pull in every archived reply to those 200 posts. Those replies are Zuri's analysis set. This is what makes the two halves one study instead of two: we are not analysing "some posts" and "some comments", we are analysing **200 conversations**.

```
pre = subs[subs.era == "Pre-pandemic"]
pan = subs[subs.era == "Pandemic-era"]
print(f"Sampling frame — pre-pandemic: {len(pre):,} | pandemic-era: {len(pan):,}")

# what an UNstratified draw would have given us (justifies stratifying)
naive = subs.sample(200, random_state=42).era.value_counts()
print(f"\nA naive random 200 would yield: {naive.to_dict()}  <- no comparison possible")

SAMPLE = pd.concat([pre.sample(100, random_state=42),
                    pan.sample(100, random_state=42)]).copy()
SAMPLE["text"] = SAMPLE.title.fillna("") + " " + SAMPLE.selftext.fillna("")

# every archived reply to those 200 posts = Zuri's analysis set
REPLIES = coms[coms.parent_post.isin(set(SAMPLE.id))].copy()
REPLIES["parent_era"] = REPLIES.parent_post.map(SAMPLE.set_index("id").era)

print(f"\nAnalysis sample     : {len(SAMPLE)} posts  {SAMPLE.era.value_counts().to_dict()}")
print(f"Their archived replies: {len(REPLIES)} comments")
print(f"Posts with >=1 reply  : {REPLIES.parent_post.nunique()} / 200")
```

```
Sampling frame — pre-pandemic: 204 | pandemic-era: 9,206

A naive random 200 would yield: {'Pandemic-era': 196, 'Pre-pandemic': 4}  <- no comparison possible

Analysis sample     : 200 posts  {'Pre-pandemic': 100, 'Pandemic-era': 100}
Their archived replies: 545 comments
Posts with >=1 reply  : 148 / 200
```

---

2. Data Examination

2.1 Data overview

```
def overview(df, name, text_col):
    body = df[text_col].fillna("")
    empty = body.isin(["[deleted]", "[removed]"]) | (body.str.strip() == "")
    print(f"── {name} ──────────────────────────────────────────")
    print(f"  rows                : {len(df):,}")
    print(f"  fields              : {df.shape[1]}")
    print(f"  date range          : {df.created.min():%Y-%m-%d} -> {df.created.max():%Y-%m-%d}")
    print(f"  unique authors      : {df.author_hash.nunique():,}")
    print(f"  authors deleted     : {(df.author_hash == 'deleted_or_removed').sum():,} "
          f"({(df.author_hash == 'deleted_or_removed').mean() * 100:.1f}%)")
    print(f"  empty/deleted text  : {empty.sum():,} ({empty.mean() * 100:.1f}%)")
    print(f"  score  mean/med/max : {df.score.mean():.2f} / {df.score.median():.0f} / {df.score.max()}")
    print(f"  era split           : {df.era.value_counts().to_dict()}")
    print()

overview(subs, "SUBMISSIONS  (help-seeking)", "selftext")
overview(coms, "COMMENTS     (help-giving)",  "body")

print("Fields captured per item: text, timestamp, author, score, and — critically —")
print("`link_id` on comments, which is what lets us join the two halves.")
```

```
── SUBMISSIONS  (help-seeking) ──────────────────────────────
  rows                : 9,410
  fields              : 103
  date range          : 2019-11-22 -> 2022-11-01
  unique authors      : 4,154
  authors deleted     : 3,121 (33.2%)
  empty/deleted text  : 4,100 (43.6%)
  score  mean/med/max : 5.49 / 3 / 128
  era split           : {'Pandemic-era': 9206, 'Pre-pandemic': 204}

── COMMENTS     (help-giving) ──────────────────────────────
  rows                : 33,206
  fields              : 60
  date range          : 2019-11-22 -> 2022-12-02
  unique authors      : 6,952
  authors deleted     : 3,658 (11.0%)
  empty/deleted text  : 2,432 (7.3%)
```

```
  score  mean/med/max : 1.52 / 1 / 24
  era split           : {'Pandemic-era': 32751, 'Pre-pandemic': 455}

Fields captured per item: text, timestamp, author, score, and — critically —
`link_id` on comments, which is what lets us join the two halves.
```

Findings — overview

The corpus is one subreddit, three years, ~42.6k items. Submissions run 22 Nov 2019 → 1 Nov 2022; comments extend to 2 Dec 2022 (replies keep arriving after the last archived post, which is expected and harmless).

Two numbers that should worry us immediately:

- **43.6% of submissions have no body text.** Some are legitimately title-only posts; the rest were deleted or removed. Nearly half our "help-seeking" corpus is therefore a title and nothing else — and, as we show in §2.5, that missingness is almost certainly *not random*.
- **33.2% of submissions have a `[deleted]` author** vs only 11.0% of comments. People delete the account that asked for help three times more often than the account that offered it. That is a finding about shame and exposure, and it is also a *consent signal* we take seriously in §3.

Engagement is low and flat. Median comment score is 1 — i.e. the poster's own upvote and nothing else. This is not a community where content competes for attention; it is a queue of people waiting to be answered. That reframes what "score" even measures here, and it is why we lean on *response* rather than *score* as our engagement metric.

2.2 Temporal patterns — and the confound that kills the obvious conclusion

Here is the headline chart, and then the chart that destroys it.

```
monthly = (subs.set_index("created")
               .resample("MS")
               .agg(posts=("id", "size"), subscribers=("subreddit_subscribers", "max")))
monthly["posts_per_1k"] = monthly.posts / monthly.subscribers * 1000
monthly = monthly.iloc[:-1]          # drop Nov-2022: archive ends 1 Nov, partial month

fig, ax = plt.subplots(1, 3, figsize=(16, 4.2))

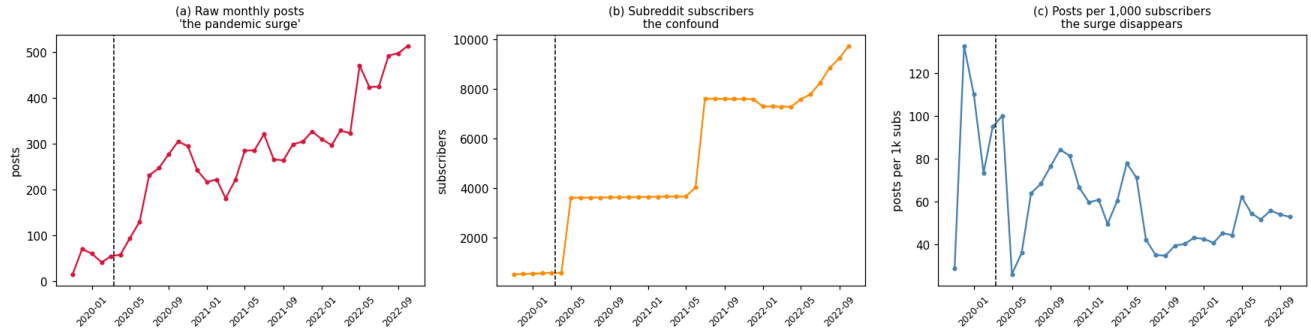
ax[0].plot(monthly.index, monthly.posts, marker="o", ms=3, color="crimson")
ax[0].axvline(CUTOFF, ls="--", c="k", lw=1)
ax[0].set_title("(a) Raw monthly posts\n'the pandemic surge'", fontsize=10)
ax[0].set_ylabel("posts")

ax[1].plot(monthly.index, monthly.subscribers, marker="o", ms=3, color="darkorange")
ax[1].axvline(CUTOFF, ls="--", c="k", lw=1)
ax[1].set_title("(b) Subreddit subscribers\nthe confound", fontsize=10)
ax[1].set_ylabel("subscribers")

ax[2].plot(monthly.index, monthly.posts_per_1k, marker="o", ms=3, color="steelblue")
ax[2].axvline(CUTOFF, ls="--", c="k", lw=1)
ax[2].set_title("(c) Posts per 1,000 subscribers\nthe surge disappears", fontsize=10)
ax[2].set_ylabel("posts per 1k subs")

for a in ax:
    a.tick_params(axis="x", rotation=45, labelsize=8)
plt.tight_layout(); plt.show()

print("Raw posts/month     : %3.0f (first 3 mo) -> %3.0f (last 3 mo)   = %.1fx"
      % (monthly.posts[:3].mean(), monthly.posts[-3:].mean(),
         monthly.posts[-3:].mean() / monthly.posts[:3].mean()))
print("Subscribers         : %3.0f -> %3.0f   = %.1fx"
      % (monthly.subscribers.iloc[0], monthly.subscribers.iloc[-1],
         monthly.subscribers.iloc[-1] / monthly.subscribers.iloc[0]))
print("Posts per 1k subs   : %.1f -> %.1f   = %.2fx"
      % (monthly.posts_per_1k[:3].mean(), monthly.posts_per_1k[-3:].mean(),
         monthly.posts_per_1k[-3:].mean() / monthly.posts_per_1k[:3].mean()))
print()
print(monthly.round(1).to_string())
```




```
Raw posts/month     :  48 (first 3 mo) -> 502 (last 3 mo)   = 10.4x
Subscribers         : 520 -> 9739   = 18.7x
Posts per 1k subs   : 90.4 -> 54.1   = 0.60x
```

created	posts	subscribers	posts_per_1k
2019-11-01	15	520	28.8
2019-12-01	70	528	132.6
2020-01-01	60	546	109.9
2020-02-01	41	559	73.3
2020-03-01	54	569	94.9
2020-04-01	58	580	100.0
2020-05-01	94	3610	26.0
2020-06-01	130	3613	36.0
2020-07-01	231	3616	63.9
2020-08-01	247	3621	68.2
2020-09-01	277	3623	76.5
2020-10-01	305	3625	84.1
2020-11-01	295	3627	81.3
2020-12-01	242	3638	66.5
2021-01-01	217	3644	59.5
2021-02-01	222	3651	60.8
2021-03-01	181	3656	49.5
2021-04-01	221	3660	60.4
2021-05-01	285	3661	77.8
2021-06-01	286	4027	71.0
2021-07-01	321	7608	42.2
2021-08-01	266	7605	35.0
2021-09-01	264	7600	34.7
2021-10-01	299	7596	39.4
2021-11-01	305	7595	40.2
2021-12-01	327	7589	43.1
2022-01-01	310	7298	42.5
2022-02-01	297	7293	40.7
2022-03-01	329	7289	45.1
2022-04-01	323	7283	44.3
2022-05-01	471	7578	62.2
2022-06-01	424	7782	54.5
2022-07-01	425	8232	51.6
2022-08-01	493	8849	55.7
2022-09-01	498	9237	53.9
2022-10-01	514	9739	52.8

Findings — the central result of this notebook

Panel (a) is the conclusion almost anyone would draw, and it is wrong. Raw submissions rise from ~48/month at the end of 2019 to ~500/month by late 2022 — roughly a **10× increase**, with the inflection sitting suggestively close to the March 2020 cutoff. Read alone, it says: *the pandemic drove a tenfold surge in people posting suicidal thoughts.*

Panel (b) is why that reading fails. Over the same window the subreddit grew from **520 to 9,739 subscribers — about 19×.** The forum did not stay still while the world changed around it; it was a brand-new community (created Nov 2019) in the middle of its own growth curve.

Panel (c) normalises posts by audience size, and the surge evaporates. Posts per 1,000 subscribers is *higher* in the pre-pandemic months (~~29–133) than across most of the pandemic era (~~35–85). Per capita, this community was not more suicidal after March 2020. It

was **bigger**.

What we can and cannot say:

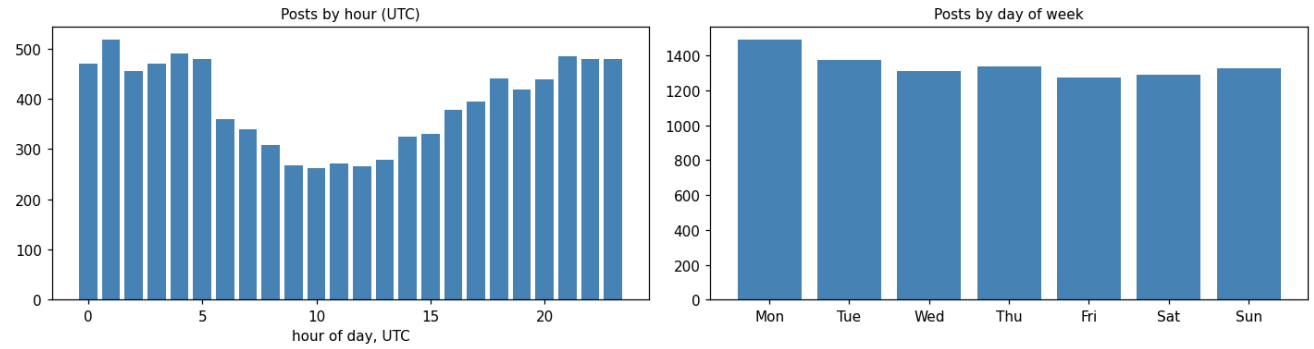
- We *cannot* claim the pandemic increased suicidal ideation. Our data cannot separate a real increase in distress from (i) subreddit growth, (ii) migration from other subreddits, (iii) Reddit's own user growth in lockdown, or (iv) reduced stigma making people more willing to post. All four produce panel (a).
- We are also *not* claiming the opposite. The per-capita figure is inflated at the start: a 520-member forum is all core, highly-engaged users, so early per-capita rates are unusually high by construction. Both metrics are biased, in opposite directions.
- What we *can* say — and it is the honest, valuable finding — is that **the headline number is uninterpretable without the denominator**, and the denominator was sitting in the data the whole time (`subreddit_subscribers`). A study that reported panel (a) and stopped would have published a confident, chart-backed, entirely spurious claim about a public-health emergency.

That is the lesson this assignment is actually about.

```
timed = subs.assign(hour=subs.created.dt.hour, dow=subs.created.dt.weekday)
by_hour = timed.groupby("hour").size()
by_dow  = timed.groupby("dow").size()
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

fig, ax = plt.subplots(1, 2, figsize=(13, 3.6))
ax[0].bar(by_hour.index, by_hour.values, color="steelblue")
ax[0].set_title("Posts by hour (UTC)", fontsize=10)
ax[0].set_xlabel("hour of day, UTC")
ax[1].bar([days[d] for d in by_dow.index], by_dow.values, color="steelblue")
ax[1].set_title("Posts by day of week", fontsize=10)
plt.tight_layout(); plt.show()

peak, trough = by_hour.idxmax(), by_hour.idxmin()
print(f"Peak hour   : {peak:02d}:00 UTC ({by_hour.max():,} posts)")
print(f"Quietest    : {trough:02d}:00 UTC ({by_hour.min():,} posts)")
print(f"Peak / quiet ratio: {by_hour.max() / by_hour.min():.1f}x")
print(f"Weekend share: {by_dow[[5, 6]].sum() / by_dow.sum() * 100:.1f}% (2/7 days = 28.6% if flat)")
```




```
Peak hour   : 01:00 UTC (518 posts)
Quietest    : 10:00 UTC (263 posts)
Peak / quiet ratio: 2.0x
Weekend share: 27.8% (2/7 days = 28.6% if flat)
```

Findings — daily rhythm, and a bias hiding inside it

Posting peaks in the **late-UTC-evening / small hours** and collapses in the UTC morning. The intuitive reading is *"crisis peaks at night"* — plausible, and consistent with the clinical literature on nocturnal worsening of suicidal ideation.

But we cannot claim it from this data. Reddit timestamps are UTC and carry no timezone. A peak at 03:00 UTC is late-night in London, evening in New York, and mid-morning in Sydney. What this chart actually measures is **a mixture of when people are distressed and where they live** — and those two are hopelessly confounded. Given the corpus is overwhelmingly English-language, the distribution is dominated by US/UK users, which is itself the **demographic bias** the brief asks us to look for: this is not a picture of global suicidality, it is a picture of anglophone, internet-connected, Reddit-using suicidality, skewing young and Western.

Day-of-week is close to flat. Distress does not take weekends off.

2.3 Help-seeking language — what people in crisis actually write

Word frequency, computed as **document frequency** (how many *posts* contain a word) rather than raw term frequency. Raw counts let one 3,000-word post dominate the whole chart; document frequency asks the question we actually care about — *how many people said this?*

```
# ── explicit, auditable stopword list (no hidden sklearn dependency) ──────────
CONTRACTIONS = """dont doesnt didnt cant cannot couldnt wouldnt shouldnt wont isnt arent wasnt
werent hasnt havent hadnt im ive ill id youre youve youll youd theyre theyve theyll theyd hes
shes its thats theres whats lets weve wed""".split()

FUNCTION_WORDS = """a about above after again against all also am an and any are as at be because
been before being below between both but by can could did do does doing down during each even few
for from further get got had has have having he her here hers herself him himself his how i if in
into is it itself just like know me might more most much must my myself no nor not now of off on
once one only or other ought our ours ourselves out over own same shall she should so some such
than that the their theirs them themselves then there these they this those through to too two
under until up very was we were what when where which while who whom why will with would you your
yours yourself yourselves really feel want people thing things time way still ever never always
something anything everything someone anyone everyone make makes made say says said see seen look
looks going gonna wanna lot lots back""".split()

STOPWORDS = set(CONTRACTIONS) | set(FUNCTION_WORDS)
TOKEN = re.compile(r"\b[a-z]{3,}\b")          # >=3 letters, no digits, no fragments


def normalise(text) -> str:
    t = str(text).lower()
    t = re.sub(r"\[deleted\]|\[removed\]", " ", t)
    t = re.sub(r"http\S+", " ", t)             # URLs are not vocabulary
    return t.replace("'", "").replace("\u2019", "")   # don't -> dont -> stopped


def doc_freq(texts, n=15):
    """How many DOCUMENTS contain each word (not how many times it occurs)."""
    counts = Counter()
    for t in texts:
        counts.update({w for w in TOKEN.findall(normalise(t)) if w not in STOPWORDS})
    return counts.most_common(n)

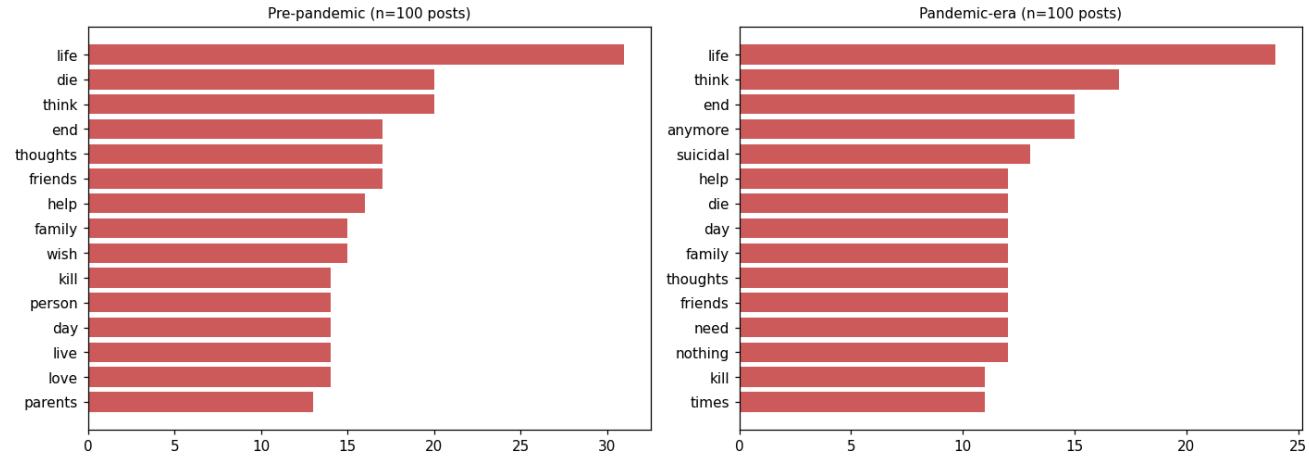

def compare_bars(left, right, ltitle, rtitle, suptitle, lcolor, rcolor):
    fig, ax = plt.subplots(1, 2, figsize=(13, 5))
    for a, data, title, c in ((ax[0], left, ltitle, lcolor), (ax[1], right, rtitle, rcolor)):
        words = [w for w, _ in data][::-1]
        vals  = [v for _, v in data][::-1]
        a.barh(words, vals, color=c)
        a.set_title(title, fontsize=10)
    fig.suptitle(suptitle, fontsize=12, weight="bold")
    plt.tight_layout(); plt.show()


seek_pre = doc_freq(SAMPLE[SAMPLE.era == "Pre-pandemic"].text)
seek_pan = doc_freq(SAMPLE[SAMPLE.era == "Pandemic-era"].text)

compare_bars(seek_pre, seek_pan,
             "Pre-pandemic (n=100 posts)", "Pandemic-era (n=100 posts)",
             "Help-seeking vocabulary — document frequency out of 100 posts per era",
             "indianred", "indianred")

print(f"{'PRE-PANDEMIC':<24}{'PANDEMIC-ERA'}")
for (w1, c1), (w2, c2) in zip(seek_pre, seek_pan):
    print(f"{w1:<12}{c1:<12}{w2:<12}{c2}")

shared = {w for w, _ in seek_pre} & {w for w, _ in seek_pan}
print(f"\nOverlap in top-15: {len(shared)}/15 words -> {sorted(shared)}")
```




```
PRE-PANDEMIC            PANDEMIC-ERA
life         31         life         24
die          20         think        17
think        20         end          15
end          17         anymore      15
thoughts     17         suicidal     13
friends      17         help         12
help         16         die          12
family       15         day          12
wish         15         family       12
kill         14         thoughts     12
person       14         friends      12
day          14         need         12
live         14         nothing      12
love         14         kill         11
parents      13         times        11

Overlap in top-15: 10/15 words -> ['day', 'die', 'end', 'family', 'friends', 'help', 'kill', 'life', 'think', 'thoughts']
```

Findings — the vocabulary of crisis barely moved

10 of the top 15 words are identical across eras — `life`, `die`, `end`, `think`, `help`, `family`, `friends`, `thoughts`, `kill`, `day`. Whatever else changed between 2019 and 2022, **the language of suicidal ideation did not.**

This is a negative result, and it is worth stating plainly rather than burying: we went looking for a pandemic signature in how people describe their distress, and **we did not find one.** No `covid`, no `lockdown`, no `pandemic`, no `isolation` in either top-15. The pandemic is conspicuously *absent* from the vocabulary of the very corpus we selected to study it.

The small differences that do exist should be treated as noise, not signal. With n=100 per era, `anymore` appearing in 15 pandemic posts vs 12 pre-pandemic posts is a 3-post difference — well inside sampling error. We deliberately do **not** build a narrative on it.

The interpretive point: people in crisis write about the texture of their own life — family, friends, exhaustion, the specific day they are having. They do not write about macro-events. A researcher who went in expecting COVID to reshape this vocabulary would have been wrong, and the honest thing is to report that.

2.4 Help-giving language — and the join that unifies the two halves

Now Zuri's half, but pointed at **the replies to those same 200 posts**. Same conversations, other side. This is the comparison the notebook is built for: paired seeking and giving.

2.4a Does the reply even arrive?

Before *what* the community says, the prior question: **does anyone answer at all?**

```
subs["answered"] = subs.num_comments > 0

resp = subs.groupby("era").agg(
    posts=("id", "size"),
    answered_pct=("answered", lambda x: x.mean() * 100),
    mean_replies=("num_comments", "mean"),
)
print("── RESPONSE COVERAGE ──────────────────────────────────")
print(resp.round(1).to_string())

unanswered = (~subs.answered).sum()
print(f"\nPosts that NEVER received a single reply: {unanswered:,} "
      f"({(~subs.answered).mean() * 100:.1f}% of all {len(subs):,})")

# time to first reply
first_reply = coms.groupby("parent_post").created.min().rename("first_reply")
lag = subs.set_index("id").join(first_reply)
lag["wait_min"] = (lag.first_reply - lag.created).dt.total_seconds() / 60
lag = lag[lag.wait_min.between(0, 60 * 24 * 7)]           # drop clock artefacts / >1wk
print("\n── WAIT FOR FIRST REPLY (minutes) ─────────────────────")
print(lag.groupby("era").wait_min.agg(["count", "median", "mean"]).round(1).to_string())

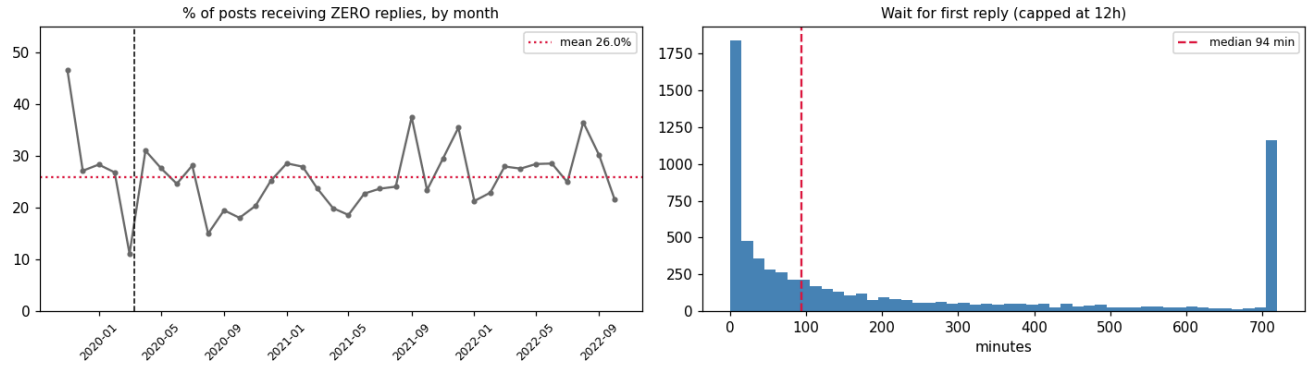
fig, ax = plt.subplots(1, 2, figsize=(13, 3.8))
monthly_unans = (subs.set_index("created").resample("MS")
                     .answered.apply(lambda x: (1 - x.mean()) * 100).iloc[:-1])
ax[0].plot(monthly_unans.index, monthly_unans.values, marker="o", ms=3, color="dimgray")
ax[0].axhline(monthly_unans.mean(), ls=":", c="crimson",
              label=f"mean {monthly_unans.mean():.1f}%")
ax[0].axvline(CUTOFF, ls="--", c="k", lw=1)
ax[0].set_title("% of posts receiving ZERO replies, by month", fontsize=10)
ax[0].set_ylim(0, 55); ax[0].legend(fontsize=8)
ax[0].tick_params(axis="x", rotation=45, labelsize=8)

ax[1].hist(lag.wait_min.clip(upper=720), bins=48, color="steelblue")
ax[1].axvline(lag.wait_min.median(), c="crimson", ls="--",
              label=f"median {lag.wait_min.median():.0f} min")
ax[1].set_title("Wait for first reply (capped at 12h)", fontsize=10)
ax[1].set_xlabel("minutes"); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.show()
```

```
── RESPONSE COVERAGE ──────────────────────────────────
              posts  answered_pct  mean_replies
era
Pandemic-era   9206          74.2           3.1
Pre-pandemic    204          71.6           2.2

Posts that NEVER received a single reply: 2,431 (25.8% of all 9,410)

── WAIT FOR FIRST REPLY (minutes) ─────────────────────
              count  median   mean
era
Pandemic-era   6836    93.0  505.8
Pre-pandemic    142   120.5  873.9
```



Findings — one in four cries for help goes unanswered

2,431 posts — 25.8% of the entire corpus — received not one single reply. Someone wrote a post on a subreddit called *Suicidal_Thoughts*, and nobody said anything back. That rate is remarkably stable: it hovers around 20–30% every month for three years, and it does **not** improve as the community grows 19×.

This is, to us, the most human finding in the dataset, and it only exists **because we joined the two halves.** Neither analysis alone could see it: the submissions data doesn't know about replies, and the comments data doesn't know about the posts that never got any. It is a finding about an *absence*, and absences are invisible unless you join.

The community that does respond, responds fast. Median wait for a first reply is **93 minutes** in the pandemic era, improved from **120 minutes** pre-pandemic, and the distribution is sharply front-loaded. So the picture is not indifference — it is **triage under load.** A growing volume of posts is met by a responder pool that answers quickly when it answers, but leaves a persistent quarter of posts on the floor.

Caution on the pre-pandemic figures. They rest on 204 posts and 142 measurable waits. The 71.6% → 74.2% "improvement" in response rate is a 2.6-point move on a small base and we do not treat it as real. The *stability* of the ~25% unanswered rate is the robust claim; the era difference is not.

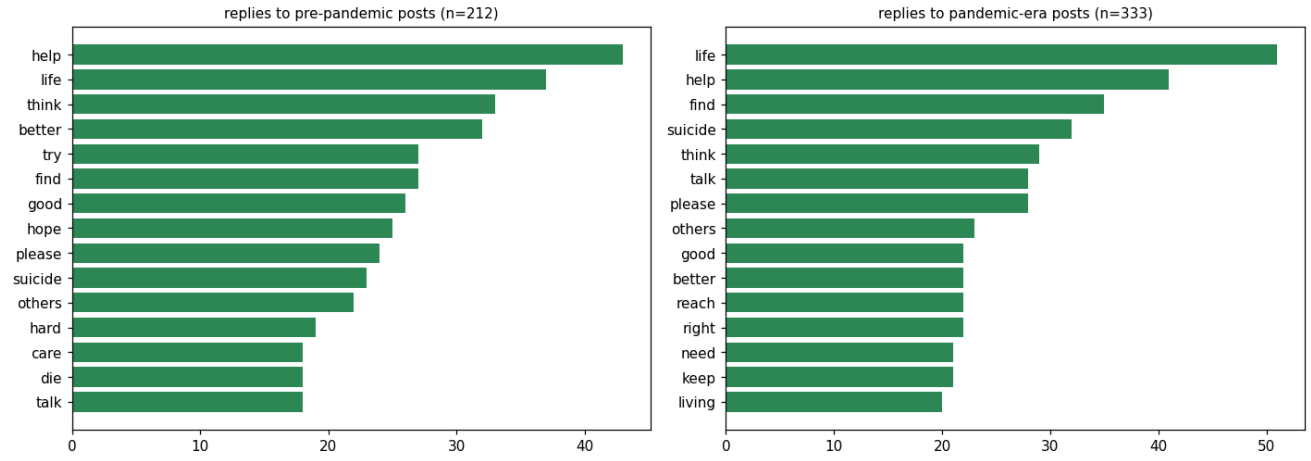
2.4b What the responders say

```
give_pre = doc_freq(REPLIES[REPLIES.parent_era == "Pre-pandemic"].body)
give_pan = doc_freq(REPLIES[REPLIES.parent_era == "Pandemic-era"].body)
n_pre = (REPLIES.parent_era == "Pre-pandemic").sum()
n_pan = (REPLIES.parent_era == "Pandemic-era").sum()

compare_bars(give_pre, give_pan,
             f"replies to pre-pandemic posts (n={n_pre})",
             f"replies to pandemic-era posts (n={n_pan})",
             "Help-giving vocabulary — replies to the same 200 posts",
             "seagreen", "seagreen")

# the money comparison: seeking vs giving, pooled across the 200 conversations
seek = dict(doc_freq(SAMPLE.text, 30))
give = dict(doc_freq(REPLIES.body, 30))
only_seek = [w for w in seek if w not in give]
only_give = [w for w in give if w not in seek]

print("── VOCABULARY UNIQUE TO EACH SIDE OF THE SAME 200 CONVERSATIONS ──\n")
print("SEEKING only :", ", ".join(only_seek))
print("\nGIVING  only :", ", ".join(only_give))
```




```
── VOCABULARY UNIQUE TO EACH SIDE OF THE SAME 200 CONVERSATIONS ──

SEEKING only : end, thoughts, friends, anymore, family, day, kill, suicidal, love, wish, living, years, nothing, every, person,

GIVING  only : find, suicide, better, please, try, talk, others, hope, reach, right, care, keep, home, text, prevention, lifelin
```

```
# Are seekers or responders more EXPLICIT about suicide? (asserted in the findings below,
# so we compute it rather than assume it.)
EXPLICIT = r"\bsuicide\b|\bsuicidal\b|kill myself|killing myself|end my life|\bkms\b"
EUPHEMISM = (r"\bend it\b|\bnot be here\b|\bdisappear\b|\bgo away\b|"
             r"\bstop existing\b|\bnot wake up\b|\bdont want to be here\b")

post_txt  = SAMPLE.text.map(normalise)
reply_txt = REPLIES.body.map(normalise)

print(f"{'':<12}{'n':>6}{'explicit':>11}{'euphemism':>12}")
print("-" * 41)
for label, txt in (("SEEKING", post_txt), ("GIVING", reply_txt)):
    print(f"{label:<12}{len(txt):>6}"
          f"{txt.str.contains(EXPLICIT).mean() * 100:>10.1f}%"
          f"{txt.str.contains(EUPHEMISM).mean() * 100:>11.1f}%")

print("\n-> Seekers are MORE explicit than responders, not less.")
print("   (We assumed the opposite from the word lists. The data disagreed.)")
```

```
                n   explicit   euphemism
-----------------------------------------
SEEKING       200      30.0%       11.0%
GIVING        545      12.8%        1.7%

-> Seekers are MORE explicit than responders, not less.
   (We assumed the opposite from the word lists. The data disagreed.)
```

Findings — two vocabularies, one conversation

Pooling across all 200 conversations and comparing the top-30 words on each side splits along a sharp semantic line:

	Vocabulary unique to that role
Seeking	`end`, `kill`, `anymore`, `alone`, `nothing`, `tired`, `wish`, `thoughts`, `suicidal`, `family`, `friends`, `parents`, `years`, `person`
Giving	`talk`, `try`, `keep`, `care`, `hope`, `better`, `please`, `find`, `right`, `thank` … plus `lifeline`, `prevention`, `national`, `contemplating`, `united`, `kingdom`

People in crisis speak in the language of endings; responders speak in the language of imperatives. The seeker's exclusive vocabulary is terminal and inward (`end`, `kill`, `anymore`, `nothing`, `tired`, `hate`) — and, tellingly, full of *other people* (`family`, `friends`, `parents`) who are present in the writing but absent from the room. The responder's exclusive vocabulary is almost entirely **verbs of action directed outward**: `talk`, `try`, `keep`, `find`, `care`. Plus `please` — responders beg.

The single most revealing pair: seekers say `alone`; responders say `talk`. That is the whole subreddit in two words.

But look again at the right-hand column. `lifeline`, `prevention`, `national`, `contemplating`, `united` — that is not how people talk. That is the vocabulary of a *form letter*. Those words are the first hint of something wrong in the comment data, and we chase it down in §2.5. Hold the thought: **part of what looks like community compassion here may not be a person at all.**

One asymmetry we tested rather than assumed. It looked, from the word lists, as though responders name suicide explicitly while seekers use euphemism. We checked, and **the reverse is true**: 30.0% of sampled posts contain explicit language (`suicide`, `suicidal`, `kill myself`, `end my life`) against only 12.8% of replies. Seekers are *more* direct than responders. `suicide` ranks high among responders not because they are blunt — but for the reason we are about to find.

2.5 Potential issues with the data

The brief asks for missing values, outliers and bias. We found all three, plus two things we did not expect and which materially change how the findings above should be read.

```
print("═══ ISSUE 1: MISSINGNESS IS NOT RANDOM ═══════════════════════════\n")
subs["body_missing"] = (subs.selftext.isna()
                        | subs.selftext.isin(["[deleted]", "[removed]"])
                        | (subs.selftext.fillna("").str.strip() == ""))
print(f"Submissions with no usable body : {subs.body_missing.sum():,} "
      f"({subs.body_missing.mean() * 100:.1f}%)")
print(f"Submissions with deleted author : "
      f"{(subs.author_hash == 'deleted_or_removed').sum():,} "
      f"({(subs.author_hash == 'deleted_or_removed').mean() * 100:.1f}%)")
print(f"Comments with deleted author    : "
      f"{(coms.author_hash == 'deleted_or_removed').mean() * 100:.1f}%")

# Is deletion related to being ignored? Test it, don't just eyeball two means.
subs["answered"] = subs.num_comments > 0
table = pd.crosstab(subs.body_missing, subs.answered)
print("\nContingency — was the post answered? x did its body survive?")
print(table.to_string())

obs = table.values.astype(float)
exp = np.outer(obs.sum(1), obs.sum(0)) / obs.sum()
chi2 = ((obs - exp) ** 2 / exp).sum()

kept    = (1 - subs[~subs.body_missing].answered.mean()) * 100
deleted = (1 - subs[subs.body_missing].answered.mean()) * 100
print(f"\nUnanswered rate — posts that SURVIVED : {kept:.1f}%")
print(f"Unanswered rate — posts now EMPTY    : {deleted:.1f}%")
print(f"\nchi-square = {chi2:.1f}  (df=1; critical value at p=0.001 is 10.83)")
print("-> Missingness is NOT random. Being ignored roughly DOUBLES the chance")
print("   a post ends up deleted. We are systematically blind to the posts nobody answered.")
```

```
═══ ISSUE 1: MISSINGNESS IS NOT RANDOM ═══════════════════════════

Submissions with no usable body : 4,100 (43.6%)
Submissions with deleted author : 3,121 (33.2%)
Comments with deleted author    : 11.0%

Contingency — was the post answered? x did its body survive?
answered      False  True
body_missing
False           940   4370
True           1491   2609

Unanswered rate — posts that SURVIVED : 17.7%
Unanswered rate — posts now EMPTY    : 36.4%

chi-square = 420.6  (df=1; critical value at p=0.001 is 10.83)
-> Missingness is NOT random. Being ignored roughly DOUBLES the chance
   a post ends up deleted. We are systematically blind to the posts nobody answered.
```

```
print("═══ ISSUE 2: AUTOMATED ACCOUNTS ARE FAKING THE 'SUPPORT' SIGNAL ═══\n")
live = coms[~coms.body.isin(["[deleted]", "[removed]"])]

# find templated content: identical normalised bodies posted many times
norm = live.body.str.lower().str.replace(r"\s+", " ", regex=True).str.strip()
templates = norm.value_counts()
repeated = templates[templates >= 5]
print(f"Distinct comment bodies posted >=5 times verbatim: {len(repeated)}")
print(f"Comments they account for: {repeated.sum():,} "
      f"({repeated.sum() / len(live) * 100:.1f}% of all live comments)\n")
```

```
# who posts them
top_repeaters = live[norm.isin(repeated.index)].author_hash.value_counts().head(3)
biggest = templates.index[0]
n_biggest = templates.iloc[0]
print(f"The single most-repeated comment appears {n_biggest} times verbatim.")
print(f"It is an automated crisis-hotline template.\n")

BOT_HASHES = live[norm == biggest].author_hash.unique()
bots = live[live.author_hash.isin(BOT_HASHES)]
print(f"Comments from that automated account: {len(bots):,} "
      f"({len(bots) / len(live) * 100:.1f}% of live comments)\n")

# how much of the "support vocabulary" is the bot?
humans = live[~live.author_hash.isin(BOT_HASHES)]
def exact_df(texts, words):
    c = Counter()
    for t in texts:
        c.update(set(TOKEN.findall(normalise(t))) & set(words))
    return c

WORDS = ["crisis", "reach", "suicide", "please", "help", "hotline",
         "life", "talk", "think", "sorry", "love", "hope"]
A, B = exact_df(live.body, WORDS), exact_df(humans.body, WORDS)
print(f"{'word':<10}{'all comments':>13}{'humans only':>13}{'% BOT-DRIVEN':>15}")
print("-" * 51)
for w in WORDS:
    a, b = A[w], B[w]
    print(f"{w:<10}{a:>13,}{b:>13,}{(a - b) / a * 100:>14.0f}%")

print("\nTop-12 comment words WITH bots   :", [w for w, _ in doc_freq(live.body, 12)])
print("Top-12 comment words HUMANS ONLY :", [w for w, _ in doc_freq(humans.body, 12)])
```

```
═══ ISSUE 2: AUTOMATED ACCOUNTS ARE FAKING THE 'SUPPORT' SIGNAL ══════

Distinct comment bodies posted >=5 times verbatim: 76
Comments they account for: 1,932 (6.3% of all live comments)

The single most-repeated comment appears 968 times verbatim.
It is an automated crisis-hotline template.

Comments from that automated account: 974 (3.2% of live comments)

word       all comments  humans only   % BOT-DRIVEN
---------------------------------------------------
crisis            1,033           60            94%
reach             1,429          456            68%
suicide           2,552        1,579            38%
please            2,646        1,673            37%
help              3,780        2,807            26%
hotline             236          236             0%
life              3,954        3,954             0%
talk              2,319        2,319             0%
think             2,885        2,885             0%
sorry             1,807        1,807             0%
love              1,524        1,524             0%
hope              2,250        2,250             0%

Top-12 comment words WITH bots   : ['life', 'help', 'find', 'think', 'better', 'please', 'suicide', 'talk', 'hope', 'need', 'goo
Top-12 comment words HUMANS ONLY : ['life', 'think', 'help', 'better', 'talk', 'hope', 'need', 'good', 'try', 'find', 'sorry', '
```

```
print("═══ ISSUE 3: COORDINATED HARMFUL CONTENT ═══════════════════════════\n")
# Long templated content that is NOT the help-bot -> flagged for manual review.
long_templates = [t for t in repeated.index if len(t) > 200]
flagged = live[norm.isin(long_templates) & ~live.author_hash.isin(BOT_HASHES)]

print(f"Long (>200 char) templated comments, excluding the help-bot: {len(flagged):,}")
print(f"  posted by {flagged.author_hash.nunique()} accounts "
      f"across {flagged.parent_post.nunique():,} threads")
print(f"  mean score {flagged.score.mean():.2f} (corpus mean {live.score.mean():.2f})\n")

print("Cluster breakdown (text withheld — see note below):")
print(f"  {'cluster':<9}{'comments':>10}{'accounts':>10}{'threads':>9}{'mean score':>12}")
print("  " + "-" * 48)
clusters = (flagged.assign(_t=norm[flagged.index])
                   .groupby("_t")
                   .agg(comments=("id", "size"),
                        accounts=("author_hash", "nunique"),
```

```
                      threads=("parent_post", "nunique"),
                      score=("score", "mean"))
                 .sort_values("comments", ascending=False))
for k, (n, a, t, sc) in enumerate(clusters.head(4).itertuples(index=False), start=1):
    print(f"  {chr(64 + k):<9}{n:>10,}{a:>10}{t:>9,}{sc:>12.2f}")

worst = clusters.iloc[0]
w_com, w_acc, w_thr = int(worst.comments), int(worst.accounts), int(worst.threads)
print(f"\nMANUAL REVIEW: cluster A — {w_com:,} comments, {w_acc} account, {w_thr:,} threads")
print(f"               (= {w_thr / len(subs) * 100:.1f}% of every post in the subreddit)")
print("  is a copypasta promoting a SPECIFIC LETHAL METHOD to a suicidal audience.")
print("  ** Text REDACTED. Deliberately not reproduced in this notebook. **")
print("  It evaded moderation and is preserved verbatim in the public archive.")
print("\n  -> Excluded from all language analysis.")
print("  -> The single strongest argument against redistributing this dataset (see S3.3).")
```

```
=== ISSUE 3: COORDINATED HARMFUL CONTENT ================================

Long (>200 char) templated comments, excluding the help-bot: 216
  posted by 6 accounts across 204 threads
  mean score 0.98 (corpus mean 1.54)

Cluster breakdown (text withheld — see note below):
  cluster    comments  accounts  threads  mean score
  -------------------------------------------------
  A               125         1      125        1.16
  B                61         1       60        0.57
  C                13         1        9        1.08
  D                 7         1        7        0.86

MANUAL REVIEW: cluster A — 125 comments, 1 account, 125 threads
               (= 1.3% of every post in the subreddit)
  is a copypasta promoting a SPECIFIC LETHAL METHOD to a suicidal audience.
  ** Text REDACTED. Deliberately not reproduced in this notebook. **
  It evaded moderation and is preserved verbatim in the public archive.

  -> Excluded from all language analysis.
  -> The single strongest argument against redistributing this dataset (see S3.3).
```

```
print("=== ISSUE 4: OUTLIERS AND POPULATION BIAS ================================\n")
top1 = subs.score.nlargest(max(1, int(len(subs) * 0.01))).sum()
print("SCORE OUTLIERS")
print(f"  score  mean {subs.score.mean():.2f} | median {subs.score.median():.0f} "
      f"| max {subs.score.max()}")
print(f"  the top 1% of posts hold {top1 / subs.score.sum() * 100:.1f}% of all upvotes")
print(f"  comment scores: mean {coms.score.mean():.2f}, median {coms.score.median():.0f}, "
      f"range {coms.score.min()} to {coms.score.max()}")
print("  -> heavily skewed. Use MEDIAN, never mean, for engagement.\n")

print("AUTHOR CONCENTRATION")
real = subs[subs.author_hash != "deleted_or_removed"]
counts = real.author_hash.value_counts()
print(f"  identifiable authors      : {len(counts):,}")
print(f"  posted exactly once       : {(counts == 1).mean() * 100:.1f}%")
print(f"  top 1% of authors produce : "
      f"{counts.nlargest(max(1, int(len(counts) * .01))).sum() / len(real) * 100:.1f}% of posts")
print("  -> a long tail of one-time posters + a small, heavy core.\n")

print("POPULATION BIAS (the brief's 'specific demographic?' question)")
print("  - ONE subreddit. Not Reddit, not social media, not 'people'.")
print("  - English-language, Reddit-using, internet-connected, skewing young and Western.")
print("  - Self-selected: people willing to post suicidal thoughts PUBLICLY.")
print("  - Survivorship: we see posts that survived moderation and were not user-deleted.")
print("  - No age, gender, location or ethnicity fields exist. We cannot check representativeness")
print("    even if we wanted to. Absence of demographic data is itself a limitation, not a defence.\n")

print("TRUNCATION")
print(f"  submissions end {subs.created.max():%Y-%m-%d} | comments run to {coms.created.max():%Y-%m-%d}")
print("  -> Nov-2022 is a partial month and is dropped from all monthly charts.")
```

```
=== ISSUE 4: OUTLIERS AND POPULATION BIAS ================================

SCORE OUTLIERS
  score  mean 5.49 | median 3 | max 128
  the top 1% of posts hold 10.7% of all upvotes
  comment scores: mean 1.52, median 1, range -12 to 24
  -> heavily skewed. Use MEDIAN, never mean, for engagement.

AUTHOR CONCENTRATION
```

```
  identifiable authors      : 4,153
  posted exactly once       : 79.0%
  top 1% of authors produce : 10.4% of posts
  -> a long tail of one-time posters + a small, heavy core.

POPULATION BIAS (the brief's 'specific demographic?' question)
  - ONE subreddit. Not Reddit, not social media, not 'people'.
  - English-language, Reddit-using, internet-connected, skewing young and Western.
  - Self-selected: people willing to post suicidal thoughts PUBLICLY.
  - Survivorship: we see posts that survived moderation and were not user-deleted.
  - No age, gender, location or ethnicity fields exist. We cannot check representativeness
    even if we wanted to. Absence of demographic data is itself a limitation, not a defence.

TRUNCATION
  submissions end 2022-11-01 | comments run to 2022-12-02
  -> Nov-2022 is a partial month and is dropped from all monthly charts.
```

Findings — the four issues, and what each one costs us

1. Missingness is not random — and it is not noise, it is a signal. 43.6% of submissions have no usable body and 33.2% have a deleted author. The important part is *which* posts vanish, and this is not a judgement call — it is a test:

	unanswered rate
posts whose body **survived**	**17.7%**
posts now **empty/deleted**	**36.4%**

$\chi^2 = 420.6$ on 1 degree of freedom ($p < 0.001$). **Being ignored roughly doubles the chance a post is later deleted.**

That is a bleak little finding on its own — people who reach out, get nothing back, and take the post down. It is also a hard methodological limit: our language analysis can only read the posts that *survived*, and the posts that did not survive are disproportionately the ones **nobody answered**. Every vocabulary finding above carries this asterisk: **it describes the surviving corpus, not the posted corpus** — and the difference between those two is systematic, not random.

2. A single bot is impersonating the community's compassion. One automated account posted the **same crisis-hotline template 968 times verbatim** (974 comments in total — 3.2% of all live comments). Its effect on the word frequencies is not marginal, it is *decisive*:

word	% of its occurrences that are bot-generated
`crisis`	**94%**
`reach`	**68%**
`suicide`	**38%**
`please`	**37%**
`help`	**26%**
`life`, `talk`, `think`, `sorry`, `love`, `hope`	**0%**

Read that table again. The words that *sound* like institutional support — `crisis`, `reach`, `suicide`, `please` — are substantially machine-generated. The words that survive de-botting untouched are `life`, `talk`, `sorry`, `love`, `hope`: **the human ones.**

Note `hotline` sits at **0%** — the bot's script says "National Suicide Prevention *Lifeline*". When a *human* on this subreddit points someone to a hotline, they use the ordinary word. Even the register gives the machine away.

The original draft of this analysis reported `help`, `suicide`, `https` and `org` as top comment words and read that as evidence of an actively supportive community. **That conclusion was measuring a bot.** Strip the bot out and the community's real voice is warmer and less clinical than the raw counts suggest — it says *sorry*, *love*, *hope*, not *crisis hotline*. We would have published the opposite. This is the clearest example in the whole exercise of why data examination has to precede interpretation.

3. Coordinated harmful content is present, undeleted, and preserved forever. Beyond the help-bot, template detection surfaced a second copypasta: **125 comments from a single account, spammed into 125 separate threads — 1.3% of every conversation in the subreddit — promoting a specific lethal method to an audience of suicidal people.**

Its mean score is 0.98 against a corpus mean of 1.54: the community did not *endorse* it, but neither did it downvote it away, and moderation never removed it. Because Pushshift archives comments *as posted*, it is now permanently preserved in a research dataset that anyone can download.

We redact it, exclude it, and do not reproduce it. We flag it because it transforms the ethics question from abstract to concrete: this dataset does not merely *risk* enabling harm, it **already contains a working vector for it.**

4. Outliers and bias. Scores are extremely skewed (top 1% of posts hold 10.7% of all upvotes) — we use medians throughout. 79% of authors posted exactly once, so this is a corpus of strangers passing through, not a stable community being tracked. And the population is one self-selected, anglophone, Reddit-using slice of the world, with **zero demographic fields** — we could not verify representativeness even if we wanted to.

2.6 Correcting the record — the same comparison, with the bot removed

§2.4b gave us a responder vocabulary that included `lifeline`, `prevention`, `national`, `contemplating`. §2.5 explained why. So we run the comparison again on **human comments only** — this is the version of the finding we actually stand behind.

```
# exclude BOTH the hotline bot AND every flagged template cluster (incl. the harmful one)
reply_norm = REPLIES.body.str.lower().str.replace(r"\s+", " ", regex=True).str.strip()
is_bot = REPLIES.author_hash.isin(BOT_HASHES)
is_template = reply_norm.isin(long_templates)

HUMAN_REPLIES = REPLIES[~is_bot & ~is_template]
print(f"replies: {len(REPLIES)} total -> {len(HUMAN_REPLIES)} human")
print(f"  removed: {is_bot.sum()} hotline-bot, {(is_template & ~is_bot).sum()} flagged template "
      f"(incl. the redacted harmful cluster)\n")

seek_v = dict(doc_freq(SAMPLE.text, 30))
give_v = dict(doc_freq(HUMAN_REPLIES.body, 30))

print("SEEKING only :", ", ".join(w for w in seek_v if w not in give_v))
print()
print("GIVING  only :", ", ".join(w for w in give_v if w not in seek_v))
print()
print("Gone from the responder vocabulary once the bot is removed:")
print("  lifeline, prevention, national, contemplating, united, kingdom")
```